

TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

SAĞLIKTA YAPAY ZEKÂ YARIŞMALARI

(Bilgisayarlı Görüyle Abdomen (Karın) Bölgesi için Hastalık
Tespiti Kategorisi)

PROJE DETAY RAPORU

TAKIM ADI: DENE-YAZ

TAKIM ID: 399878

İÇİNDEKİLER

	<u>Sayfa</u>
KAPAK	1
İÇİNDEKİLER	2
1. Proje Mevcut Durum Değerlendirmesi (20 puan)	3
2. Özgünlük (30 puan)	4
3. Sonuçlar ve İnceleme (30 puan)	8
4. Deney ve eğitim aşamalarında kullanılan veri setleri (15 puan)	10
5. Referanslar (5 puan)	10

ŞEKİLLER

	<u>Sayfa</u>
Şekil 1. Aşama 1 Deneysel Model Mimarimiz	4
Şekil 2. Aşama 1 için kullanılan yöntem şeması	5
Şekil 3. U-Net Mimarisi	6
Şekil 4. Aşama 2 Deneysel Model Mimarimiz	6
Şekil 5. Aşama 2 için kullanılan yöntem şeması	7
Şekil 6. Aşama 1 için 50 epoch sonunda doğruluk ve kayıp değerleri	8
Şekil 7. Aşama 2 için 20 epoch sonunda doğruluk ve kayıp değerleri	8
Şekil 8. Örnek test görüntüsü	9
Şekil 9. BT görüntülerinden çıkarılan gerçek ve tahmin edilen maske görüntüleri	9

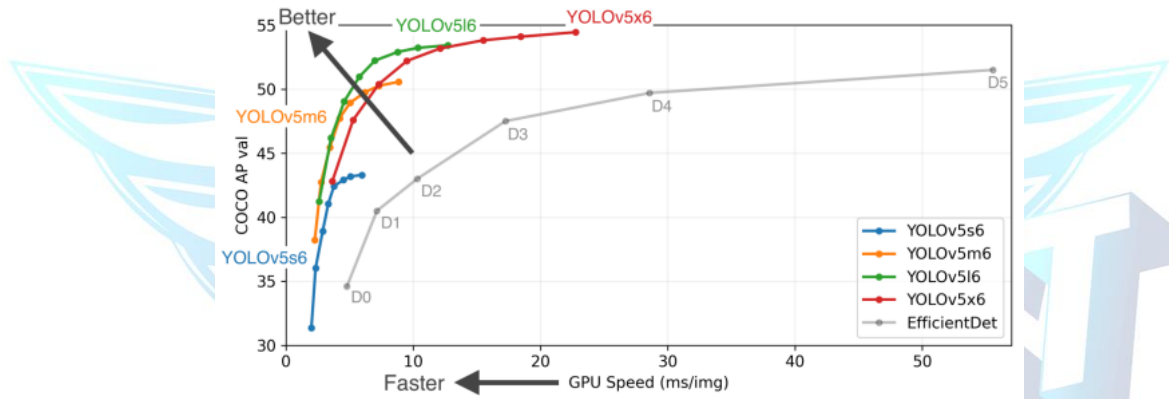
1. Proje Mevcut Durum Değerlendirmesi (20 puan)

Ön Tasarım aşamasında EfficientNet, YOLOv4, RetinaNet ve Detectron algoritmaları detaylıca araştırıldı. Araştırmalar esnasında YOLOv5 algoritması ile karşılaşıldı. YOLOv5'in bahsi geçen algoritmalarından en başarılısı olarak bilinen EfficientNet ile başarı oranlarında yakın değerlere sahip olmasına rağmen daha hızlı sonuç verdiği öğrenilmiş ve bundan dolayı model eğitimi YOLOv5 ile EfficientNet üzerine yoğunlaştırılmıştır.

Takımlara sağlanan geçmiş yıllara ait veri setinin yeterli olmaması ve sınıfların homojen dağılmaması sebebiyle yeni veri setleri araştırılmış ve her bir Abdomen(karın) Bölgesi için Hastalıkları için ayrı ayrı veri setine ulaşılmıştır. Bu veri setleri kullanılarak modellerin eğitilmesi tamamlanmıştır.

YOLOv5'in özelleştirilmiş modellerin en iyileştirmesi için sunduğu probleme özel hiper parametre optimizasyonu denenmiş ve uygun parametreler ile YOLOv5 modelleri eğitime başlanmıştır. EfficientNet ile eğitimler tamamlanarak test sonuçları, YOLOv5 modellerinin sonuçları ile karşılaştırılmıştır.

YOLOv5 [7] ve EfficientNet günümüzde yüksek başarı oranları ile öne çıkmaktadır. Başarı oranlarındaki yakınlık sebebiyle sürekli birbirleriyle karşılaştırılmakta ve hız olarak YOLOv5 öne çıkmaktadır.



Yazılım mimarisi olarak PyTorch seçilmiştir. PyTorch, Torch kütüphanesine dayanan açık kaynaklı bir makine öğrenme kütüphanesidir. Bilgisayarla görme ve doğal dil işleme gibi uygulamalar için kullanılır. Öncelikle Facebook'un AI Araştırma laboratuvarı (FAIR) tarafından geliştirilmiştir. Kullanımı kolay API sağlaması nedeniyle çalıştırılması basit bir yapıya sahiptir.

TensorFlow, Theano, Caffe ve CNTK gibi çerçevelerin çoğu statik bir dünya görüşüne sahiptir. Bir sinir ağı kurmalı ve aynı yapı tekrar tekrar kullanılmalıdır. Sinir ağının davranış şeklini değiştirmek, kişinin sıfırdan başlaması gerektiği anlamına gelir. PyTorch, sıfır gecikme ile ve ek yük ile ağırların davranış şeklini istediğiniz gibi değiştirmenize olanak tanıyan, ters modlu otomatik farklılaşma, adı verilen bir teknik kullanmaktadır. Bu teknik PyTorch'a özgü olmasa da, bugüne kadarki en hızlı uygulamalarından biridir. Modelde en iyi hızı ve esnekliği elde etmeyi amaçlar [8].

PyTorch, tek bir back-end kullanmak yerine CPU, GPU ve çeşitli işlevsel özellikler için farklı back-end kullanır.

Dilimleme, indeksleme, matematik işlemleri, lineer cebir, indirgeme gibi bilimsel hesaplama ihtiyaçlarınızı hızlandırmak ve bunlara uymak için çok çeşitli tensör rutinleri sunmaktadır.

PyTorch, büyük bulut platformlarında da iyi bir şekilde desteklenir ve önceden oluşturulmuş görüntüler aracılığıyla sorunsuz geliştirme ve kolay ölçeklendirme, GPU'larda büyük ölçekli eğitimi ve daha fazlasını sağlar [9].

2. Özgünlük (30 puan)

Derin Öğrenme modelleri geliştirilmesinde sınıf başına düşen etiket sayısının 100.000 adet üzerinde olmasının ve sınıf başına homojen veri dağılımının önemini biliyoruz.

Yarışmanın teknik şartnamesini sağlayan, karın bölgesi organlarına ait veri setlerine araştırma yaparak ulaşılabilmektedir ancak yarışma kapsamındaki tespit edilecek nesne türlerinden sınıflarının yarışmaya özel olmasından ötürü diğer sınıflarla eşit veri sayısına ulaşmadan aynı model içerisinde kullanılması yönelimli bir eğitime yol açacaktır. Bu yüzden sadece ilgili sınıftan sorumlu daha küçük bir veri setiyle ayrı bir model eğitmeyi planladık.

Hiper parametreler, eğitimin çeşitli yönlerini kontrol eder ve onlar için en uygun değerlerini bulmak zor olabilir. Izgara aramaları gibi geleneksel yöntemler, yüksek boyutlu arama uzayı, boyutlar arasındaki bilinmeyen korelasyonlar ve her noktada uygunluğu değerlendirmenin, birbiriyle bağımlı parametrelerin oluşturduğu olasılık uzayının büyüklüğü nedeniyle çabucak inatçı hale gelebilir ve Genetik Algoritma'yı (GA) hiper parametre aramaları için uygun bir aday yapar.

YOLOv5, GA yı kullanarak özelleştirilmiş modellerin en iyileştirmesi için probleme özel hiper parametre optimizasyonu sunar [11]. Biz de YOLOv5' in bu opsiyonunu kullanarak modelimize özel hiper parametreleri belirleyip eğitimlerimizi yapmayı planladık.

Object detection modelleri genelde küçük objeleri bulmakta zorlanıyor. O yüzden geliştirilen farklı farklı çözümler mevcut. Bunlardan biri YOLO için kullandığımız SPP modülüdür (Spatial Pyramid Pooling) [12]. SPP modülü, Yolo'nun evrimsel (convolutional) katmanlarından sonra üretilen öznetelikleri (feature) farklı ölçeklerde toplayarak (max pooling ile), obje algılayan katmanlara devredilir.

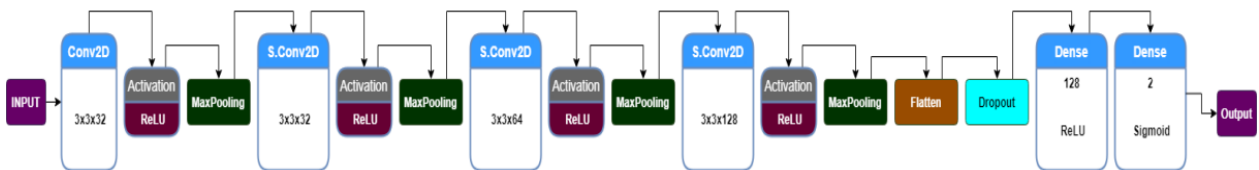
Modeli test ederken, bazen false positive durumu olduğunu fark ettik. Fakat bu false positivelere boyutu normal objelerden çok daha küçüktü. Örneğin, yerdeki bir taşı insan gibi algılama durumu mevcuttu. Bunu önlemek için bir "alpha" hiperparametresi eklendi. Eğer $[(\text{algılanan objenin boyutu})/(\text{görüntünün boyutu}) < \alpha]$ ise, o zaman o obje algılanmış sayılmaz. Test videolarında alpha için bulunan en iyi değer %0.045 olarak tespit edilmiştir.

Nesnelerin algılanmasında kareler(frame) arasında bazen oluşan tespit edememe (false negative) sorununun çözümü için Kalman filtresinin kullanılmasına karar verildi. Kalman filtresi yardımıyla nesnenin önceki karelerdeki hareket bilgisi kullanılarak nesne tespit edilemeyen karelerdeki nesne pozisyonunun tahmin edilmesi planlandı.

Kalman filtresinin istenen sonucu vermemesi durumunda, Kalman filtresine alternatif metod olarak, aradaki yanlış negatif kareleri doldurmak için objelerin pozisyonunun gelecek kareler ve geçmiş karelerdeki pozisyonun ortalaması alınarak hesaplanabileceği düşünüldü.

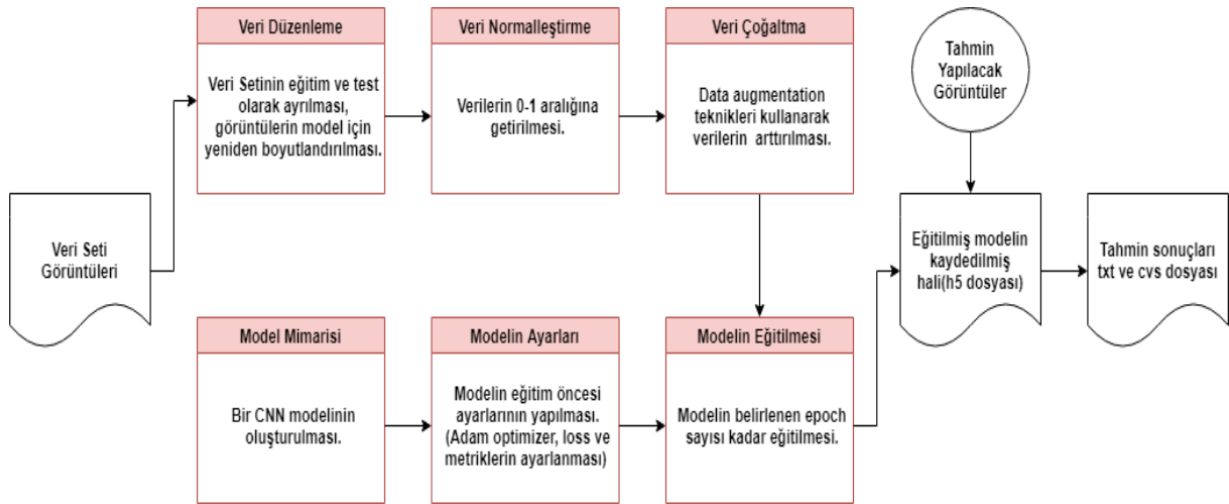
2.1. Aşama 1 için kullanılan yöntem ve mimari

Yarışmanın birinci aşaması için Python dilinde kendi yazdığımız bir Convolutional Neural Networks (CNN) [2] modeli kullanmaya karar verdik. Evrimsel Sinir Ağları (CNN) görüntü sınıflandırma işlemi için çeşitli katmanlardan oluşan yapılar kullanılır [3]. Birkaç katmandan oluşan bu modelimizi kullanarak yarışmanın birinci aşamasındaki sınıflandırma problemini çözmeyi hedefledik. Bu modeli oluşturmak için Google'ın ücretsiz ve açık kaynak olarak sunduğu Tensorflow [4] sistemini kullanmaya karar verdik. Tensorflow sistemi içinde barındırdığı birçok kütüphane ile kolay model oluşturmaya olanak sağlayan bir yapıdır. Biz modelimizi oluştururken güçlü bir model oluşturma kütüphanesi olan Keras [5] kütüphanesini kullanmayı tercih ettik. Deney ve testlerimiz için birkaç farklı model oluşturarak deneyler yaptık. Aşağıda oluşturduğumuz deneysel modelin mimarisi görülmektedir.



Şekil 1. Aşama 1 Deneysel Model Mimarimiz

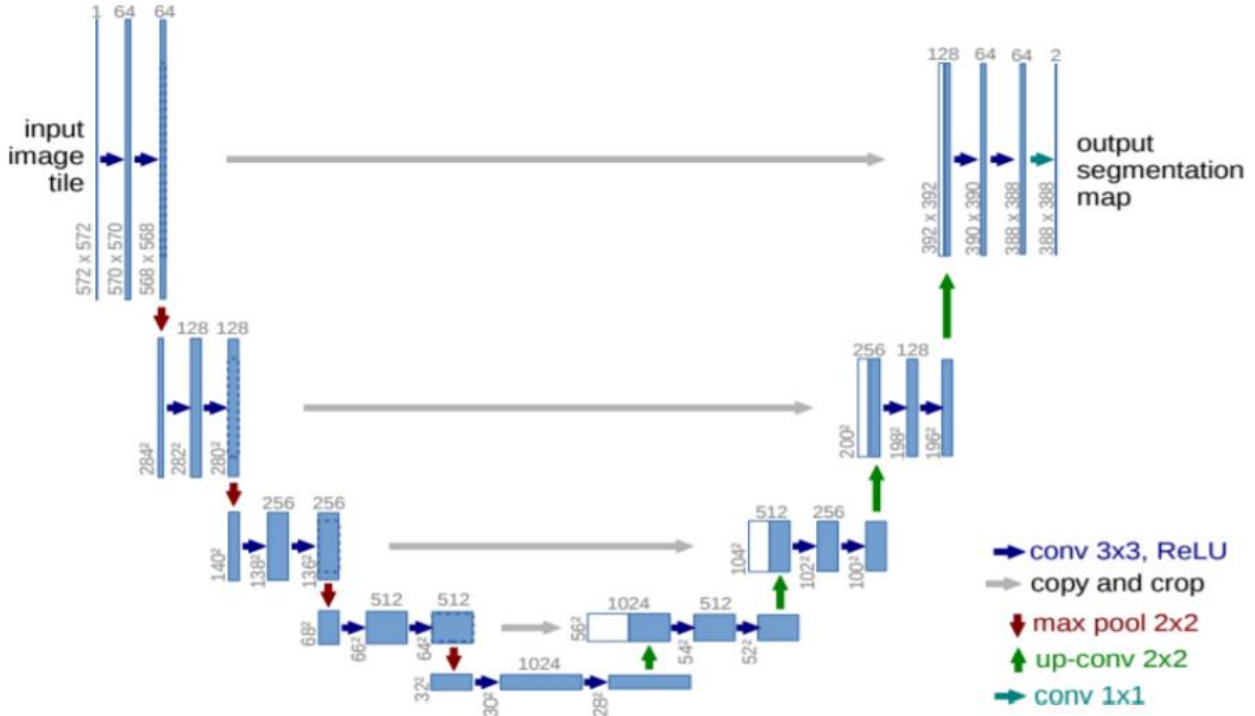
Deneysel modelimiz 1 adet 32 filtreli Convolutional katmanı, 3 adet sırası ile 32, 64 ve 128 filtreli Separable Convolution [6] katmanı, 1 adet Flatten(düzleştirici) katman, 1 adet 128 yapay nörondan oluşan Dense katmanı ve son katman olarak 2 çıkış veren çıkış katmanından oluşmaktadır. Modelimiz de katmanlar arasında aktivasyon fonksiyonu olarak ReLu [7] aktivasyon fonksiyonu kullanılmıştır. Son çıkış katmanının da Sigmoid fonksiyonu uygulanmıştır. Ayrıca Convolutional katmanları arasında MaxPooling [8] işlemi uygulanmıştır. Flatten katmanı ile Dense katmanı arasında Dropout [9] işlemi yapılmıştır. Bu katmanlara ek olarak veri setini çoğaltmak için data augmentation [10] işlemi input katmanına uygulanmıştır. Deneysel modelimizde yaklaşık olarak 6 milyonun üzerinde parametre işlenmektedir. Modelimizi eğitmek için veriler modele verilmeden önce bazı işlemlerden geçirilmiştir. Öncelikle veriler keras kütüphanesi yardımı ile %80 eğitim ve %20 test verisi olacak şekilde ayrılmıştır. Daha sonra veriler eğitimin sistemlerimiz de daha hızlı çalışması için 320x320 çözünürlükte yeniden boyutlandırılmıştır. Ayrıca veriler kullanılmadan önce 0-1 aralığına normalleştirilmiştir. Veri setini büyütmek için data augmentation [10] işlemi kullanılarak veri sayımız artırılmıştır. Modelin eğitim sırasın da elde ettiği en iyi ağırlık değerlerinin saklanması için eğitim boyunca en iyi ağırlık değerleri kayıt edilmiştir. Ayrı eğitim tamamlandığında tüm model h5 dosyası formatında kayıt edilerek farklı dosyalar üzerinden kullanım imkanı sağlanmıştır. Model eğitimi başlatılmadan önce derleme işlemi için gerekli ayarlamalar yapılmıştır. Optimizasyon yöntemi olarak Adam optimizer [11] kullanılmıştır. Loss(kayıp) fonksiyonu olarak Categorical Cross Entropy [12] fonksiyonu kullanılmıştır. Doğruluk metriği olarak Accuracy [13] metriği kullanılmıştır. Bu doğruluk hesaplama yöntemin de tahminlerin etiketlere ne sıklıkla eşit olduğunu hesaplanır. Modelin eğitimi tamamlandıktan sonra yarışma sırasında verilecek olan görüntüler üzerinde toplu tahmin yapacak bir python dosyası yazdık. Bu dosya eğitilmiş modeli kullanarak bir klasörde bulunan tüm görüntüler için tahmin oluşturduktan sonra sonuçları bir tablo halinde yarışma isterine uygun olarak txt ve cvs dosyası olarak kayıt etmektedir. Şekil 2 de görülen şemada yarışmanın 1. aşaması için kullanılacak yöntem açıklanmıştır.



Şekil 2. Aşama 1 için kullanılan yöntem şeması

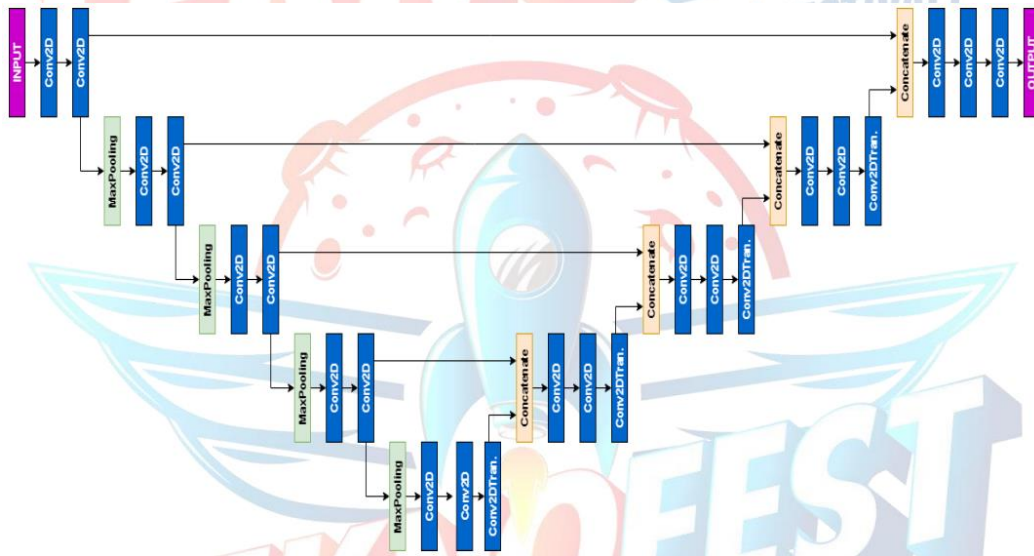
2.2. Aşama 2 için kullanılan yöntem ve mimari

Bu aşamada görüntü segmentasyon probleminin çözümü için U-Net [14] mimarisini temel alacak bir model oluşturup kullanmaya karar verdik. Bu modeli oluşturmak için Google'ın ücretsiz ve açık kaynak sistemi olan Tensorflow [4] ve Keras [5] kütüphanesini kullandık. U-Net mimarisinde girişte verilen görüntüler çıkışta segmente(maske) edilmiş olarak alınmaktadır. U-Net ismini U harfine benzeyen mimarisinden almaktadır. Şekil 3'deki görselde U-Net mimarisi görülmektedir.



Şekil 3. U-Net Mimarisi

U-Net mimarisinin çıkışında bir tamamen bağlı katman (fully connected) katmanı bulunmamaktadır. Sadece evrişimli (convolutional) katmanlardan oluşmaktadır. Aktivasyon fonksiyonu olarak katmanlar arasında ReLu [7] aktivasyon fonksiyonu kullanılmıştır. U-Net mimarisinin 2 temel yapısı vardır. Birinci aşama da sınır bölgesinde bulunan pikseller görüntülerin çevresine eklenerek büyütülür. İkinci yarıda ters evrişim işlemi başlar ve görüntü çıkışta orijinal boyutuna tekrar getirilir. Yarışmanın bu aşaması için kullandığımız mimari şekil 4'deki şemada gösterilmiştir.



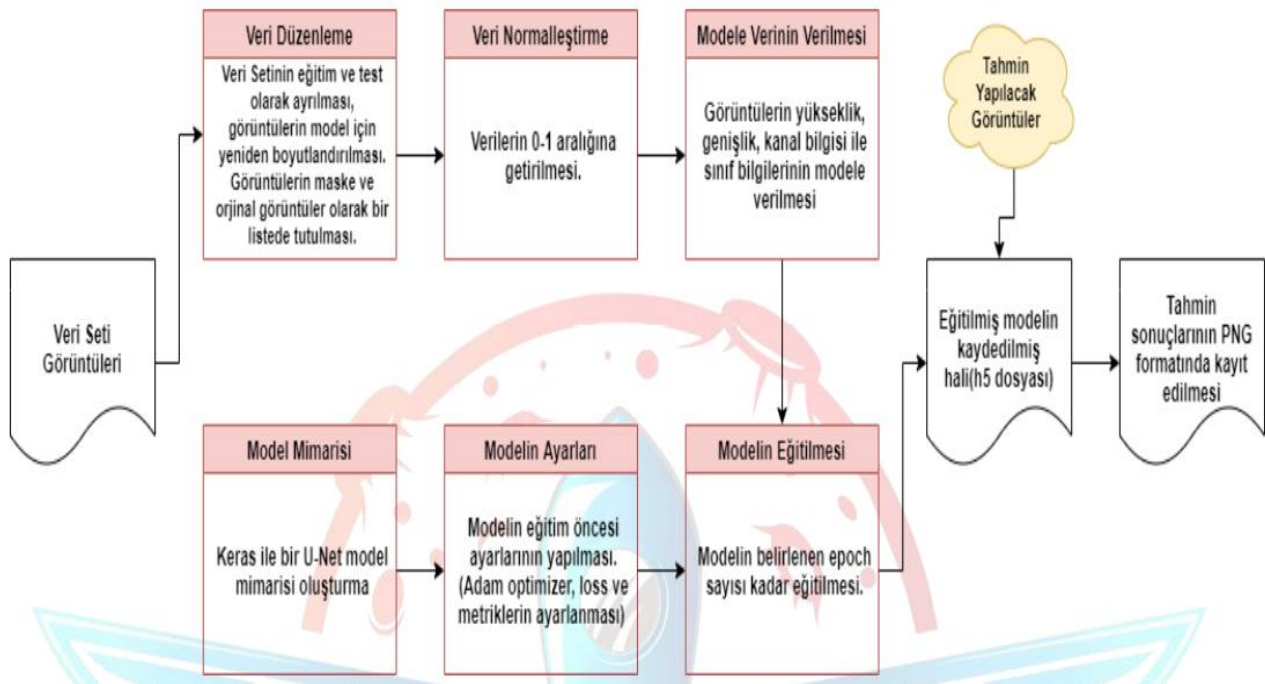
Şekil 4 Aşama 2 Deneysel Model Mimarimiz

Modelimiz de evrişimli (convolutional) katmanlar arasında aktivasyon fonksiyonu olarak ReLu[7] aktivasyon fonksiyonu kullanılmıştır. Çıkış katmanından önce Softmax aktivasyon fonksiyonu uygulanmıştır. Ayrıca her evrişimli (convolutional) katmandan sonra dropout [9] işlemi uygulanmıştır.

Modeli eğitmek için kullanılacak görüntüler ve onlara ait maske görüntüleri modele verilmeden önce bazı işlemlerden geçirilmiştir. Öncelikle veriler yeniden boyutlandırılarak (256x256) numpy kütüphanesi yardımı ile array (dizi) haline getirilip bir listede tutulmuştur. Veriler eğitim ve test verisi olarak Keras kütüphanesi yardımı ile ayrılmıştır. Daha sonra keras kütüphanesi kullanarak veriler eğitim için normalleştirilmiştir. Ayrıca veriler 3 sınıf olarak şekilde Keras kütüphanesi kullanılarak sınıflara ayrılmışlardır.

Model eğitimi başlatılmadan önce derleme işlemi için gerekli ayarlamalar yapılmıştır. Optimizasyon yöntemi olarak Adam optimizer [11] kullanılmıştır. Loss(kayıp) fonksiyonu olarak Categorical Cross Entropy [12] fonksiyonu kullanılmıştır. Doğruluk metriği olarak Accuracy [13] metriği kullanılmıştır. Eğitim tamamlandığında tüm model h5 dosya formatında kayıt edilmiştir. Toplu tahmin işlemi için ayrı bir python dosyası hazırlanmıştır. Tahmin işlemlerinden sonra tahmin maskeleri ayrı klasörde PNG görüntü formatında kayıt edilmiştir.

Şekil 5’deki şemada yarışmanın 2.aşaması için kullanılacak yöntem açıklanmıştır.

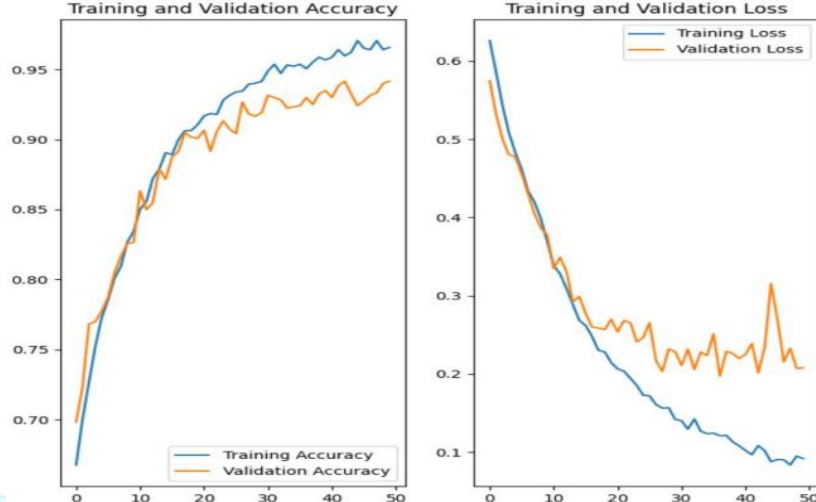


Şekil 5. Aşama 2 için kullanılan yöntem şeması

3. Sonuçlar ve İnceleme (30 puan)

3.1. Aşama 1 için deneysel sonuçlar

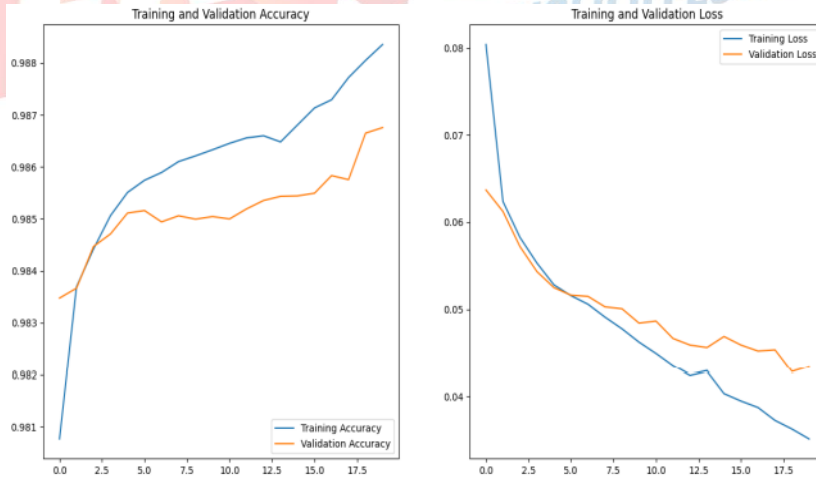
Aşama 1 için oluşturmuş olduğumuz deneysel CNN modelinin doğruluk oranı şu aşamada %90 ve üzerindedir. Loss (kayıp) değerlerimiz ise beklentinin biraz altında ve 0.20 civarında çıkmaktadır. Doğruluk oranlarını yükseltmek ve loss değerlerini düşürmek için yarışma gününe kadar deneysel modeli geliştirmeye devam edeceğiz. Şekil 6'daki görsel de modelimizin doğruluk ve kayıp değerleri görülmektedir.



Şekil 6 Aşama 1 için 50 epoch sonunda doğruluk ve kayıp değerleri

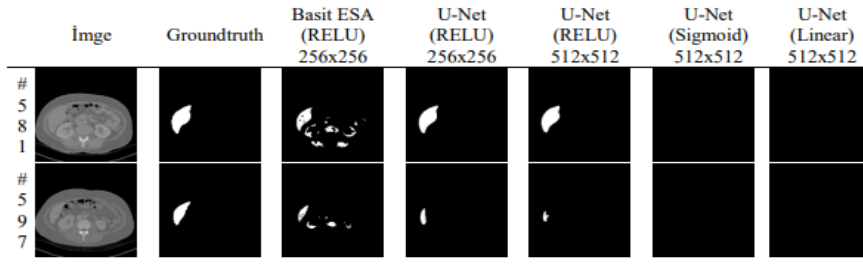
3.2. Aşama 2 için deneysel sonuçlar

Aşama 2 için oluşturmuş olduğumuz U-Net deneysel modelin %90'nın üzerinde doğruluk değerine sahip olduğu görülmüştür. Fakat U-Net mimarisinden dolayı görüntüler piksel piksel işlendiğinden %90 doğruluğa rağmen bazen modelden çok farklı tahmin sonuçları alınabilmektedir. Modelin geliştirilmesi yönünde çalışmalar yarışma gününe kadar sürdürülecektir. Modelin düşük eğitim sürecinde bile başarılı sonuçlar verdiğini göstermek için eğitim süreci 20 epoch ile sınırlandırılmıştır. Şekil 7'de modelin 20 epoch eğitim sonucunda elde ettiği doğruluk ve kayıp durumu görülmektedir.

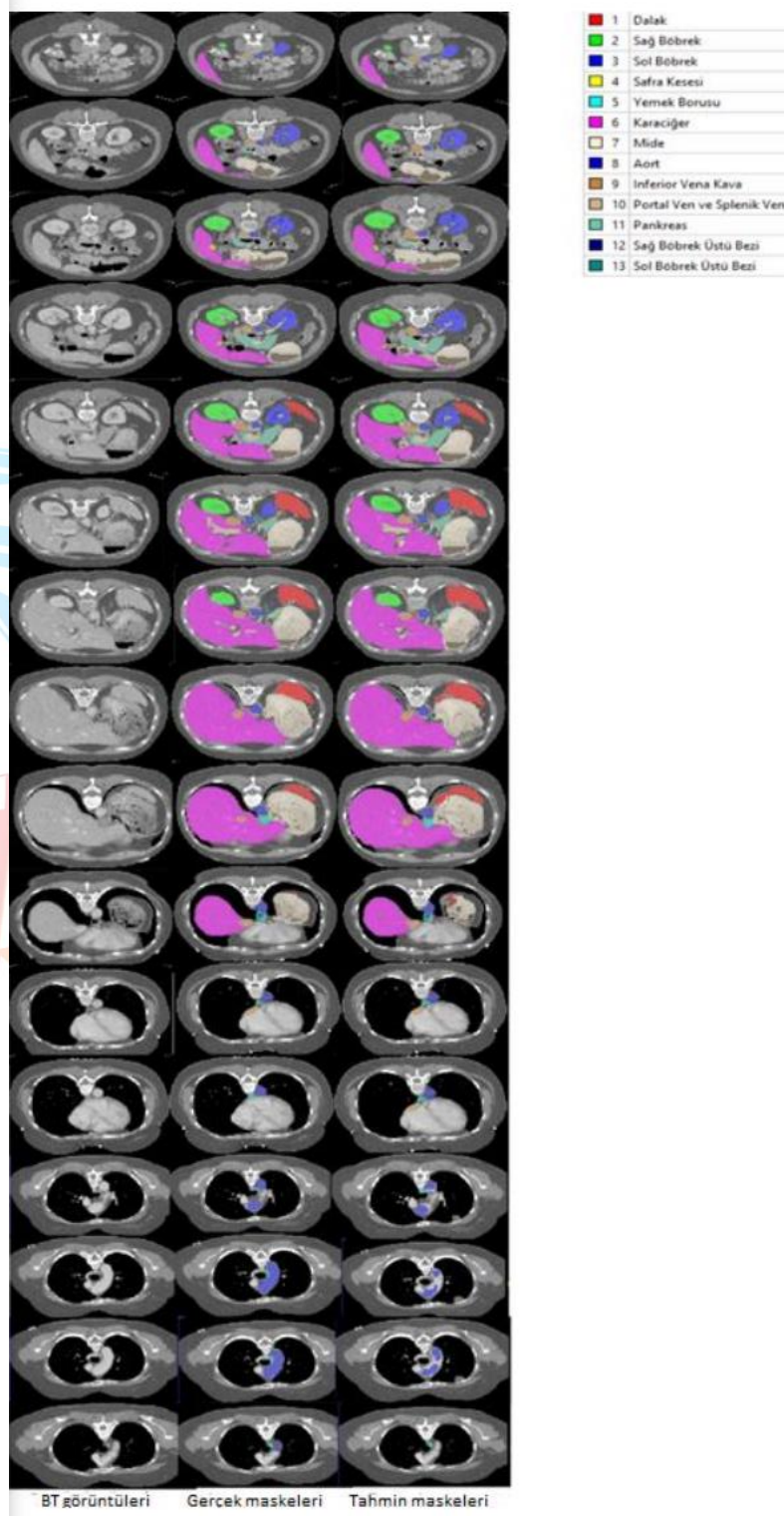


Şekil 7. Aşama 2 için 20 epoch sonunda doğruluk ve kayıp değerleri

Eğitilmiş modeli test etmek için daha önce modele verilmemiş bazı BT görüntüleri modele verilmiştir. Test sonuçları aşağıdaki şekillerde gösterilmiştir.



Şekil 8. Örnek test görüntüsü



Şekil 9. BT görüntülerinden çıkarılan gerçek ve tahmin edilen maske görüntüleri

4. Deney ve eğitim aşamalarında kullanılan veri setleri (15 puan)

Model eğitimlerimiz için Sağlık Bakanlığı tarafından paylaşılan veri seti kullanılmıştır. Bu verisetinin kullanımı için Sağlık Bakanlığı tarafından sağlanan kurumsal gizlilik taahhütnamesi takımımız içindeki tüm üyelerimiz tarafından imzalanarak belge PDF formatında Sağlık Bakanlığına gönderilmiştir.

Sağlık Bakanlığı tarafından sağlanan verilerin içerisinde hastaların Bilgisayarlı Tomografi (BT) görüntüleri paylaşılmıştır. Bu görüntüler hem DICOM hem de PNG formatında olup model eğitimlerinde istenilen formatta eğitim sağlanabilmektedir.

Biz modelimiz de PNG formatında görüntü dosyalarını kullandık.

Bakanlığın sağladığı veri setinde görüntüler 512x512 piksel boyutlarında ve 32 bit derinliğindedir. Geliştirmiş olduğumuz deneysel modeller için şu aşamada ek veri setine ihtiyaç duymadığımız için model eğitimlerimiz de farklı bir veriseti kullanmadık.

5. Referanslar (5 puan)

- [1]. Deo, R. C. (2015). Machine Learning in Medicine, Basic Science for Clinicians, 1920-1930.
- [2]. Osisanwo, F. Y., Akinsola, J. E. T., Awodele, O., Hinmikaiye, J. O., Olakanmi, O. ve Akinjobi, J. (2017). Supervised Machine Learning Algorithms: Classification and Comparison, International Journal of Computer Trends and Technology (IJCTT), 48 (3), 128-138.
- [3]. Hu, Z., Tang, J., Wang, Z., Zhang, K., Zhang, L., Sun, Q., (2018), Deep learning for image-based cancer detection and diagnosis –A survey, Pattern Recognition, 83,134-149.
- [4]. Liu, S., Wang, Y., Yang, X., Lei, B., Liu, L., Xiang Li, S., Wang, T. (2019). Deep Learning in Medical Ultrasound Analysis: A Review., Engineering. 261-275
- [5]. Selvikvåg Lundervold, A., & Lundervold, A. (2018). An overview of deep learning in medical imaging focusing on MRI. Zeitschrift Für Medizinische Physik. 102-127
- [6]. Janney, S. S., & Chakravarty, S. (2019). Deep learning in medical and surgical instruments. Bioelectronics and Medical Devices, 833–855.
- [7]. <https://github.com/ultralytics/yolov5> , Erişim Tarihi: 06.01.2022.
- [8]. <https://github.com/pytorch/pytorch> , Erişim Tarihi: 06.01.2022.
- [9]. <https://pytorch.org/features> , Erişim Tarihi: 08.01.2022.
- [10]. NVIDIA: <https://developer.nvidia.com/cuda-gpus#compute>. Erişim Tarihi: 08.01.2022.
- [11]. <https://github.com/ultralytics/yolov5/issues/607> , Erişim Tarihi: 18.01.2022.
- [12]. Huang, Zhanchao & Wang, Jianlin. (2019). DC-SPP-YOLO: Dense Connection and Spatial Pyramid Pooling Based YOLO for Object Detection.